Programming in Python
IDC-212
Lecture-2

August, 2025

Contents

1. Data Type

1.1 List
1.2 Tuple
1.3 Set
1.4 Dictionary

Data Type

List Data Type (1/3)

When to use List Data Type?

- To represent a group of values as a single entity.
- When insertion order must be preserved.
- When duplicates are allowed.
- When heterogeneous objects are required.
- Lists are growable in nature.

Properties of Lists:

1. Insertion order is preserved.
2. Heterogeneous objects are allowed.
3. Duplicates are allowed.
4. Growable in nature.
5. Values should be enclosed in square brackets `[ ]`.

Example:

```python
list1 = [10, 10.5, 'iiser', True, 10]
print(list1)    # [10, 10.5, 'iiser', True, 10]
```

List Data Type (2/3

Indexing and Slicing:

```python
list1 = [10, 20, 30, 40]
print(list1[0])      # 10
print(list1[-1])     # 40
print(list1[1:3])    # [20, 30]

list1[0] = 100
for i in list1:
    print(i)
# Output: 100, 20, 30, 40
```

List Data Type (3/3)

Growable Nature:

```
list2.remove(20) print(list2)  [10, 30, 'iiser']
list2 = list2 * 2 print(list2)  [10, 30, 'iiser', 10, 30, 'iiser']
```

Note: Lists are ordered, mutable, heterogeneous collections that allow duplicates.

List Memory Layout Example

Python List Example: `list1 = [10, 'iiser', True]`

10	'iiser'	True
0	1	2

Each element can be accessed using its **index**.
Example: `list1[1]` $\Rightarrow$ `'iiser'`

List Indexing with Positive and Negative Indices

Example List: `[10, "A", "B", 20, 30, 10]`

-6	-5	-4	-3	-2	-1
10	A	B	20	30	10
0	1	2	3	4	5

Note: List objects are **mutable**, i.e., we can change their content.

Creation of List Objects (2/2)

Creating a List with Dynamic Input

```
list3 = eval(input("Enter List: "))
print(list3)
print(type(list3))
```

Example Input:

```
Enter List: [1, 2, 'iiser', True]
```

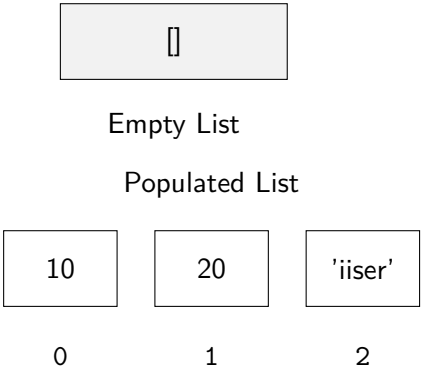
Output:

```
[1, 2, 'iiser', True]
<class 'list'>
```

Creation of List Objects with Memory Layout

Example Lists

- Empty list: `list1 = []`
- Populated list: `list2 = [10, 20, 'iiser']`



Creating List Objects with list() Function

Using list() with range():

```python
l = list(range(0, 10, 2))
print(l)
print(type(l))
```

Output:

```
[0, 2, 4, 6, 8]
<class 'list'>
```

List Slicing Examples

Syntax: list2 = list1[start : stop : step]

- **start** → Index where slice begins (default = 0)
- **stop** → Index where slice ends (default = len(list))
- **step** → Increment value (default = 1)

```python
n = [1,2,3,4,5,6,7,8,9,10]

print(n[2:7:2])    # [3, 5, 7]
print(n[4::2])     # [5, 7, 9]
print(n[3:7])      # [4, 5, 6, 7]
print(n[8:2:-2])   # [9, 7, 5]
print(n[4:100])    # [5, 6, 7, 8, 9, 10]
```

Forward Slicing in Python

Example Code:

```python
n = [1,2,3,4,5,6,7,8,9,10]
print(n[2:7:2])
# Output: [3, 5, 7]
```

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

Output: [3,5,7]

Reverse Slicing in Python

Example Code:

```python
n = [1,2,3,4,5,6,7,8,9,10]
print(n[8:2:-2])
# Output: [9, 7, 5]
```

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

Output: [9,7,5]

List vs Mutability

Key Idea: Once we create a List object, we can **modify its content**. Hence, **List objects are mutable**.

```
n = [10,20,30,40]
print(n)        # Output: [10, 20, 30, 40]

n[1] = 777
print(n)        # Output: [10, 777, 30, 40]
```

Explanation

- Lists in Python are mutable. - After creating a list, we can change its elements by index. - Example: The second element (20) was replaced with 777.

Traversing a List using `while` Loop

Definition: The sequential access of each element in a list is called **traversal**.

Using `while` Loop:

```python
numbers = [5, 10, 15, 20, 25]
index = 0
while index < len(numbers):
    print(numbers[index])
    index += 1
```

Output:

5
10
15
20
25

Traversing a List using for Loop

Using for Loop:

```python
fruits = ["apple", "banana", "cherry", "mango"]
for item in fruits:
    print(item)
```

Output:

```
apple
banana
cherry
mango
```

Comparison

- while loop requires manual index handling. - for loop directly accesses each element.

Important List Functions (1/2)

1) len(): Returns the number of elements in the list.

```
numbers = [12, 24, 36, 48, 60]
print(len(numbers))   # Output: 5
```

2) count(): Returns the number of occurrences of a specified item.

```
marks = [80, 90, 80, 75, 90, 80]
print(marks.count(80))   # Output: 3
print(marks.count(75))   # Output: 1
print(marks.count(100))  # Output: 0
```

Important List Functions (2/2)

3) index(): Returns the index of the first occurrence of the specified item.

```python
cities = ["Delhi", "Paris", "London", "Paris", "Tokyo"]

print(cities.index("Paris"))   # Output: 1
print(cities.index("Tokyo"))   # Output: 4

# Before using index(), check if element exists
print("New York" in cities)    # Output: False
```

Note

If the element is not in the list, `index()` raises a `ValueError`. Always check membership with `in` before using `index()`.

Manipulating List Elements (1/2)

1) append() Function: Adds an element to the end of the list.

```
fruits = []
fruits.append("Mango")
fruits.append("Apple")
fruits.append("Orange")
print(fruits)
```

Output: ['Mango', 'Apple', 'Orange']

Example: Add all numbers up to 50 divisible by 7

```
multiples = []
for i in range(51):
    if i % 7 == 0:
        multiples.append(i)
print(multiples)
```

Output: [0, 7, 14, 21, 28, 35, 42, 49]

Manipulating List Elements (2/2)

2) insert() Function: Inserts an element at a specified index.

```python
numbers = [11, 22, 33, 44]
numbers.insert(2, 99)   # insert at index 2
print(numbers)
```

Output:

```
[11, 22, 99, 33, 44]
```

Another Example:

```python
nums = [5, 15, 25]
nums.insert(0, 100)    # at the beginning
nums.insert(-1, 200)   # before the last element
print(nums)
```

Output:

```
[100, 5, 15, 200, 25]
```

List Function: `remove()`

Purpose:

- Removes the **first occurrence** of a specified item from the list.
- If the element is not found, Python raises a `ValueError`.

Example:

```python
n = [5, 10, 15, 10, 20]
print(n)            # [5, 10, 15, 10, 20]
n.remove(10)
print(n)            # [5, 15, 10, 20]
if 100 in n:
    n.remove(100) # This won't execute
else:
    print("100 not found in list")
```

Note: Use the `in` operator before calling `remove()` to avoid errors.

List Function: pop()

Purpose:

- Removes and returns an element from the list.
- By default, removes the last element.
- You can specify an index to remove an element from that position.

Examples:

```python
n = [10, 20, 30, 40]
print(n.pop())     # 40
print(n)           # [10, 20, 30]

print(n.pop(1))    # 20
print(n)           # [10, 30]

empty = []
# print(empty.pop()) -> IndexError: pop from empty list
```

Note: pop() is commonly used for stack operations (LIFO: Last In First Out).

List Function: `reverse()`

Purpose:

- Reverses the order of elements in a list.
- It modifies the original list in place.

Example:

```python
numbers = [11, 25, 37, 49]
numbers.reverse()
print(numbers)   # [49, 37, 25, 11]
```

Note: Unlike slicing `[::-1]`, `reverse()` does not create a copy but changes the list itself.

List Function: sort()

Purpose:

- Sorts the list in place according to natural ordering.
- For numbers: ascending order by default.
- For strings: alphabetical order by default.

Examples:

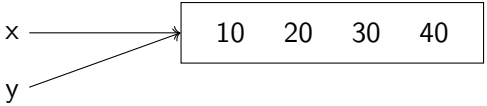
```python
nums = [42, 7, 19, 3, 15]
nums.sort()
print(nums)   # [3, 7, 15, 19, 42]

fruits = ["Mango", "Orange", "Apple", "Banana"]
fruits.sort()
print(fruits) # ['Apple', 'Banana', 'Mango', 'Orange']
```

Note: Use `sort(reverse=True)` for descending order.

Aliasing of List Objects

Aliasing is the process of giving another reference variable to the same list.

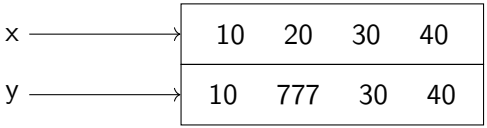
```
x = [10, 20, 30, 40]
y = x
print(id(x))
print(id(y))
y[1] = 777
print(x)  # Output: [10, 777, 30, 40]
```



Cloning of List Objects

Cloning creates a duplicate independent object using slicing or copy().

```
x = [10, 20, 30, 40]
y = x[:]    # or y = x.copy()
y[1] = 777
print(x)  # Output: [10, 20, 30, 40]
print(y)  # Output: [10, 777, 30, 40]
```



Membership Operators in Python

- Membership operators check if an element exists in a list:
  1. **in** operator
  2. **not in** operator

```python
n = [5, 15, 25, 35]
print(15 in n)
print(50 in n)
print(25 not in n)
print(100 not in n)
```

Nested Lists

We can take one list inside another list. Such lists are called **nested lists**.

Example

```python
n = [5, 15, [25, 35]]
print(n)          # [5, 15, [25, 35]]
print(n[0])       # 5
print(n[2])       # [25, 35]
print(n[2][0])    # 25
print(n[2][1])    # 35
```

Note: Nested lists can be accessed using index notation similar to multi-dimensional arrays.

Nested List as Matrix

In Python, we can represent matrices using nested lists.

Example

```python
matrix = [[2,4,6],[8,10,12],[14,16,18]]
print(matrix)

print("Row-wise output:")
for row in matrix:
    print(row)

print("Matrix style output:")
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        print(matrix[i][j], end=' ')
    print()
```

Output

Output:

[2, 4, 6 , [8, 10, 12], [14, 16, 18]]

- Row-wise output:

2, 4, 6
8, 10, 12
14, 16, 18

- Matrix style output:
  - 2 4 6
  - 8 10 12
  - 14 16 18

List Comprehensions

List Comprehensions provide an easy and compact way to create lists from any iterable (like List, Tuple, Dictionary, or Range) with optional conditions.

Syntax:

$$\text{list} = [\text{expression for item in iterable if condition}]$$

Examples

```python
# Squares of numbers 1 to 8
squares = [x*x for x in range(1,9)]
print(squares)   # [1, 4, 9, 16, 25, 36, 49, 64]
# Powers of 3 for 1 to 5
powers = [3**x for x in range(1,6)]
print(powers)    # [3, 9, 27, 81, 243]1
# Even numbers from squares
evens = [x for x in squares if x % 2 == 0]
print(evens)     # [4, 16, 36, 64]
```

Tuple Basics

Definition:

- Tuple is similar to List, except it is **immutable**.
- Once created, contents cannot be modified.
- Also called a **read-only list**.
- Insertion order is preserved.
- Allows duplicates and heterogeneous objects.
- Positive indexing: starts from 0.
- Negative indexing: starts from -1.
- Represented using `()`.

Tuple Creation Examples

```python
t = (12, 24, 36, 48)
print(t)            # (12, 24, 36, 48)
print(type(t))      # <class 'tuple'>

t = ()
print(type(t))      # <class 'tuple'>

t = (15,)
print(t)            # (15,)
print(type(t))      # <class 'tuple'>
```

Note: A trailing comma is mandatory for single-valued tuples.

Accessing Tuple Elements

```python
t = (5, 10, 15, 20, 25)
print(t[0])        # 5
print(t[-1])       # 25

t = (5, 10, 15, 20, 25, 30)
print(t[1:4])      # (10, 15, 20)
print(t[2:])       # (15, 20, 25, 30)
print(t[::2])      # (5, 15, 25)
```

Tuple vs Immutability

```
t = (10, 20, 30, 40)
t[1] = 70          # TypeError: 'tuple' object
                   # does not support item assignment
```

Mathematical Operations on Tuples

```python
t1 = (1, 2, 3)
t2 = (4, 5, 6)
t3 = t1 + t2
print(t3)          # (1, 2, 3, 4, 5, 6)

t1 = (7, 8)
t2 = t1 * 3
print(t2)          # (7, 8, 7, 8, 7, 8)
```

Important Functions of Tuple

```python
t = (12, 24, 12, 36, 24)
print(len(t))          # 5
print(t.count(24))     # 2
print(t.index(36))     # 3

t = (40, 15, 25, 10)
print(sorted(t))                  # [10, 15, 25, 40]
print(sorted(t, reverse=True))    # [40, 25, 15, 10]
```

Sorting a Tuple in Python

- In Python, using `sorted()` on a tuple returns a **list**, not a tuple.
- Reason: `sorted()` always produces a mutable sequence (list).
- To convert it back to a tuple, use `tuple()`.

Example

```python
t = (40, 15, 25, 10)
# Sorted result as list
res1 = sorted(t)
print(res1)           # [10, 15, 25, 40]
print(type(res1))     # <class 'list'>
res2 = sorted(t, reverse=True)
print(res2)           # [40, 25, 15, 10]
print(type(res2))     # <class 'list'>
res3 = tuple(sorted(t)) # Converting back to tuple
print(res3)           # (10, 15, 25, 40)
print(type(res3))     # <class 'tuple'>
```

Important Functions of Tuple

```python
t = (9, 4, 12, 7)
print(min(t))        # 4
print(max(t))        # 12

a, b, c = (11, 22, 33)
print("a=", a, "b=", b, "c=", c)
# a= 11 b= 22 c= 33
```

Packing and Unpacking in Tuples

```python
# Packing
t = 10, 20, 30, 40
print(t)            \# (10, 20, 30, 40)

# Unpacking
a, b, c, d = t
print(a)            \# 10
print(b)            \# 20
print(c)            \# 30
print(d)            \# 40

# Using * for variable-length unpacking
x, *y = t
print(x)            \# 10
print(y)            \# [20, 30, 40]
```

Tuple Comprehension

```
t = (x**2 for x in range(1,5))
print(type(t))       # <class 'generator'>
for val in t:
    print(val)       # 1 4 9 16
```

Generator vs Tuple Comprehension

What is a Generator?

- Using () with a comprehension creates a generator, not a tuple.
- A generator evaluates elements **on demand** (lazy evaluation).
- Saves memory for large sequences.

Code Example:

```
t = (x**2 for x in range(1,5))
print(type(t))          # <class 'generator'>
for val in t:
    print(val)          # 1 4 9 16
```

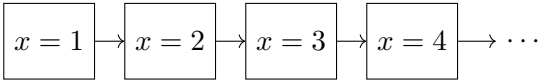
To convert to tuple:

```
t = tuple(x**2 for x in range(1,5))
```

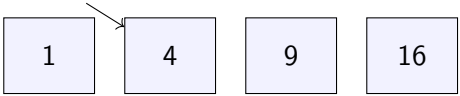
Generator vs Tuple

Memory Behavior: Generator vs Tuple

Generator:



Tuple:



Generator: Computes one value at a time
Tuple: All values computed and stored at once

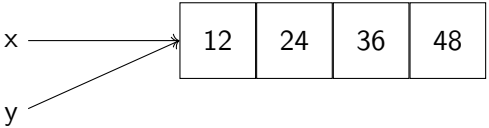
List vs Tuple

List	Tuple
Values inside `[]`	Values inside `()`
Mutable	Immutable
Contents can change	Contents cannot change
Not hashable as dict keys	Hashable, can be dict keys
Use if content changes	Use if content is fixed

Tuple Memory Representation and Immutability

Tuples are Immutable:

- Once created, tuple elements cannot be changed.
- Multiple variables can point to the same tuple, but the data remains fixed.



Immutable: Values cannot be changed

Tuples and Immutability (1/2)

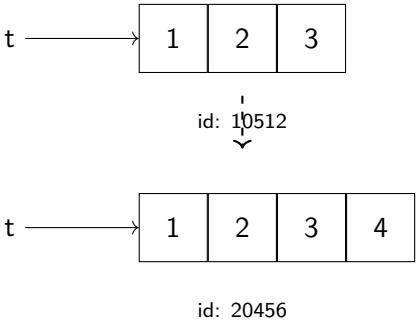
Key Idea:

- Tuples are **immutable**.
- Any "modification" creates a **new tuple** with a new memory address.

Example

```python
t = (1, 2, 3)
print(id(t))        # e.g., 10512

t = t + (4,)
print(id(t))        # Different id, e.g., 20456
```

Tuples and Immutability (2/2)



Note: A new tuple is created with a new memory id.

Modifying List Inside a Tuple

Key Idea: Tuples are immutable, meaning their structure cannot change. However, if a tuple contains a mutable object (like a list), we **can modify the contents of that object**.

```python
# Tuple containing a list
t = (10, 20, [30, 40])

# Modifying the list inside the tuple
t[2][0] = 99
t[2].append(50)

print(t)
# Output: (10, 20, [99, 40, 50])
```

Explanation:

- The tuple `t` is immutable → we cannot add/remove tuple elements.
- The list inside `t` is mutable → we can change its contents.

Tuple Containing a List (1/2)

Concept: Tuples are immutable, but if a tuple contains a mutable object (like a list), we can still modify that list.

```python
# Tuple with a list inside
t = (10, 20, [30, 40])

# Modify the list inside the tuple
t[2][0] = 99
t[2].append(50)

print(t)     # (10, 20, [99, 40, 50])
```

Introduction to Sets

- A Set is a collection of **unique values** in Python.
- **Duplicates:** Not allowed.
- **Insertion order:** Not preserved (though elements can be sorted).
- **Indexing/Slicing:** Not supported.
- **Data type:** Heterogeneous elements allowed.
- **Mutability:** Sets are mutable; their elements can be changed.
- **Representation:** Elements are enclosed within curly braces.
- **Operations:** Support union, intersection, difference, etc.

Creating Set Objects

Example 1:

```
s = {15, 25, 35, 45}
print(s)                # {35, 15, 45, 25}
print(type(s))          # <class 'set'>
```

Example 2:

```
l = [10, 20, 30, 20, 10]
s = set(l)
print(s)                # {10, 20, 30}
```

Creating Set with set() Function

Example:

```
s = set(range(4))
print(s)            # {0, 1, 2, 3}
```

Empty Set Special Case:

```
s = {}
print(type(s))      # <class 'dict'>

s = set()
print(type(s))      # <class 'set'>
```

add() and update() Functions

add(): Add one element

```
s = {10, 20}
s.add(30)
print(s)    # {10, 20, 30}
```

update(): Add multiple elements

```
s = {5, 15}
s.update([25, 35], range(3))
print(s)    # {0, 1, 2, 35, 5, 15, 25}
```

copy(), pop(), remove(), discard(), clear()

copy():

```
s = {10, 20}
s1 = s.copy()
print(s1)      # {10, 20}
```

pop():

```
s = {40, 50, 60}
print(s.pop())   # Random element removed
print(s)
```

remove(), discard(), clear()

remove(): Raises error if element not present

```
s = {10, 20, 30}
s.remove(20)
print(s)    # {10, 30}
```

discard(): No error if element missing

```
s.discard(40)
print(s)    # {10, 30}
```

clear(): Removes all elements

```
s.clear()
print(s)    # set()
```

Mathematical Operations on Sets (1/2)

Union:

```
x = {10, 20, 30}
y = {30, 40, 50}
print(x | y)        # {10, 20, 30, 40, 50}
```

Intersection:

```
print(x & y)        # {30}
```

Mathematical Operations on Sets (2/2)

Difference:

```
print(x - y)        # {10, 20}
print(y - x)        # {40, 50}
```

Symmetric Difference:

```
print(x ^ y)        # {10, 20, 40, 50}
```

Membership Operators

```python
s = set("iiser")
print('i' in s)     # True
print('z' in s)     # False
```

Set Comprehensions

```python
s = {x*x for x in range(6)}
print(s)    # {0, 1, 4, 9, 16, 25}
```

```python
s = {2**x for x in range(2, 8, 2)}
print(s)    # {4, 16, 64}
```

Indexing and Applications

Indexing is not allowed in sets:

```
s = {10, 20, 30}
print(s[0])    # Error
```

Application: Removing Duplicates

```
l = [10, 20, 10, 30]
s = set(l)
print(s)       # {10, 20, 30}
```

Finding Vowels in a Word

```
w = "iiserth"
s = set(w)
v = {'a', 'e', 'i', 'o', 'u'}
print(s & v)        # {'i', 'e'}
```

Lab Exercise: Biodiversity Study using Sets

Problem Statement: Two forests (Region A and Region B) were surveyed for plant species. Using Python sets, perform the following tasks:

1. Create sets for species found in each region.
2. Find the species common to both regions.
3. Find the species unique to Region A.
4. Find all species found across both regions.

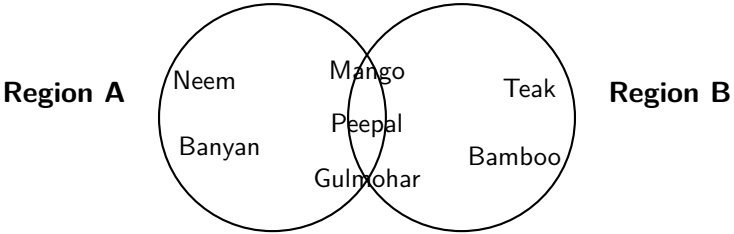
Sample Data:

```
regionA = {"Neem", "Banyan", "Mango", "Peepal", "Gulmohar"}
regionB = {"Mango", "Teak", "Bamboo", "Peepal", "Gulmohar"}
```

Expected Output for the Biodiversity Study

```
# Common species
print(regionA & regionB)
# {'Mango', 'Peepal', 'Gulmohar'}

# Unique to Region A
print(regionA - regionB)
# {'Neem', 'Banyan'}

# All species across both regions
print(regionA | regionB)
# {'Neem', 'Banyan', 'Mango', 'Peepal',
#  'Gulmohar', 'Teak', 'Bamboo'}
```

Biodiversity Study Visualization



Note:

- Intersection: Mango, Peepal, Gulmohar
- Unique to Region A: Neem, Banyan
- Unique to Region B: Teak, Bamboo

Lab Exercise: Climate Data Analysis using Lists

Problem Statement: Meteorologists recorded daily maximum temperatures of a week. Analyze the data using lists.

```python
temps = [32, 34, 31, 29, 33, 30, 35]

print("Average:", sum(temps)/len(temps))  # 32.0
print("Max Temp:", max(temps))            # 35
print("Min Temp:", min(temps))            # 29

temps.sort()
print("Sorted:", temps)                   # [29,30,31,32,33,34,35]

# Correcting a value
temps[2] = 28
print("After correction:", temps)
```

Lab Exercise: Seismic Sensor Locations using Tuples

Problem Statement: Seismologists place sensors at fixed GPS coordinates. Represent each sensor's location as a tuple and analyze them.

```python
# Tuple for each sensor location (latitude, longitude)
sensor1 = (19.0760, 72.8777)   # Mumbai
sensor2 = (28.7041, 77.1025)   # Delhi
sensor3 = (13.0827, 80.2707)   # Chennai
sensors = (sensor1, sensor2, sensor3)
print("All Sensors:", sensors)
print("First Sensor:", sensors[0])
# (19.0760, 72.8777)
# Count how many times a location repeats
print(sensors.count(sensor1))
# Attempting to modify a sensor's location
sensors[0] = (20.0, 75.0)
# TypeError: 'tuple' object does not support item assignment
```

Introduction to Dictionaries

- A Dictionary stores data as **key-value pairs**.
- Keys must be unique, values can be duplicated.
- Keys and values can be heterogeneous.
- Dictionaries are mutable and dynamic.
- Indexing and slicing are not supported.
- Also known as:
  - `Map` in C++/Java
  - `Hash` in Perl/Ruby

Creating Dictionaries

```
# Empty dictionary
d = {}
print(d)                    # {}
print(type(d))              # <class 'dict'>

# Adding entries dynamically
d[101] = "Newton"
d[102] = "Einstein"
d[103] = "Tesla"
print(d)
# {101: 'Newton', 102: 'Einstein', 103: 'Tesla'}

# Predefined dictionary
d = {201: "Galileo", 202: "Feynman", 203: "Curie"}
print(d)
# {201: 'Galileo', 202: 'Feynman', 203: 'Curie'}
```

Accessing Dictionary Elements

```python
d = {201: "Galileo", 202: "Feynman", 203: "Curie"}

print(d[201])    # Galileo
print(d[203])    # Curie

# Handling missing keys
print(d.get(204, "Not Found"))
# Not Found
```

Student Records Example

```python
records = {"Newton": 88, "Tesla": 92, "Curie": 95}
print("Name", "\t", "Marks")
for student in records:
    print(student, "\t", records[student])
# Output:
# Name      Marks
# Newton    88
# Tesla     92
# Curie     95
```

Updating Dictionaries

```python
d = {1: "Planck", 2: "Bohr"}
print(d)
# {1: 'Planck', 2: 'Bohr'}

# Adding new entry
d[3] = "Maxwell"
print(d)
# {1: 'Planck', 2: 'Bohr', 3: 'Maxwell'}

# Updating existing entry
d[1] = "Heisenberg"
print(d)
# {1: 'Heisenberg', 2: 'Bohr', 3: 'Maxwell'}
```

Deleting Elements from Dictionary

```python
d = {1: "Planck", 2: "Bohr", 3: "Maxwell"}
print(d)
# {1: 'Planck', 2: 'Bohr', 3: 'Maxwell'}

# Delete specific key
del d[1]
print(d)
# {2: 'Bohr', 3: 'Maxwell'}

# Clear dictionary
d.clear()
print(d)
# {}
```

Important Functions of Dictionary

```
d = {1: "Curie", 2: "Raman", 3: "Hawking"}

print(len(d))          # 3
print(d.keys())        # dict_keys([1, 2, 3])
print(d.values())      # dict_values(['Curie','Raman','Hawking'])
print(d.items())       # dict_items([(1,'Curie'),(2,'Raman'),(3,'
    Hawking')])

print(d.get(2))        # Raman
print(d.pop(3))        # Hawking
print(d)               # {1: 'Curie', 2: 'Raman'}

print(d.setdefault(4, "Einstein"))
# Einstein
print(d)
# {1: 'Curie', 2: 'Raman', 4: 'Einstein'}
```

Dictionary Comprehensions

```
squares = {x: x*x for x in range(1,6)}
print(squares)
# {1:1, 2:4, 3:9, 4:16, 5:25}

cubes = {x: x**3 for x in range(1,4)}
print(cubes)
# {1:1, 2:8, 3:27}
```

Iterating with items() in Dictionary

Example: Printing Keys and Values Together

```python
scientists = {1: "Newton", 2: "Curie", 3: "Tesla"}

for key, value in scientists.items():
    print("ID:", key, "-> Name:", value)

# Output:
# ID: 1 -> Name: Newton
# ID: 2 -> Name: Curie
# ID: 3 -> Name: Tesla
```

Note: `items()` returns each entry as a (key, value) tuple.

Lab Exercise: Element Atomic Numbers

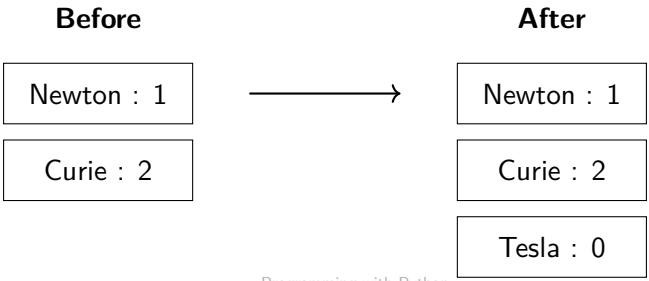
Problem Statement: Chemists need to maintain a quick lookup of atomic numbers for elements. Use a dictionary to store element names and their atomic numbers. Then print each element with its atomic number.

```python
elements = {"Hydrogen": 1,
    "Helium": 2,
    "Oxygen": 8,
    "Iron": 26,
    "Uranium": 92
}
for element, atomic_no in elements.items():
    print(f"{element} has atomic number {atomic_no}")
# Output: # Hydrogen has atomic number 1
# Helium has atomic number 2
# Oxygen has atomic number 8
# Iron has atomic number 26
# Uranium has atomic number 92
```

Working of setdefault()

Example: Adding Scientist Discoveries

```python
discoveries = {"Newton": 1, "Curie": 2}
# Key 'Tesla' not present, so add with default 0
discoveries.setdefault("Tesla", 0)
print(discoveries)
# Output:
# {'Newton': 1, 'Curie': 2, 'Tesla': 0}
```



Thank You!

Questions?